

BIVORITA

Trabajo Integrador Final - Programación 1

Juego de la serpiente (Snake) implementado en lenguaje C

Integrantes: Jenaro Ceccacci · Federico Camerini · Julián Panin Suárez

5 de julio de 2026

Índice

1. Introducción.....	3
2. Integrantes y participación en el trabajo.....	3
3. Descripción general del funcionamiento del programa	4
3.1 Flujo del programa.....	4
3.2 Entorno de desarrollo y compilación.....	4
4. Especificación y justificación de las estructuras de datos utilizadas	5
4.1 Matriz de tablero.....	5
4.2 Serpiente (lista enlazada de Nodo)	5
4.3 Manzana y Obstáculo.....	5
4.4 Jugador (lista enlazada, ranking).....	6
4.5 Complejidad y eficiencia	6
5. Detalle de las funciones principales implementadas	6
5.1 Módulo snake.c	6
5.2 Módulo tablero.c	7
5.3 Módulo visuales.c	7
5.4 Módulo teclado.c	7
5.5 Módulo jugadores.c	8
5.6 Módulo main.c	8
6. Dificultades encontradas durante el desarrollo y cómo fueron abordadas	8
7. Conclusión	9
8. Anexo: controles y funcionamiento del ranking	9

1. Introducción

Este informe documenta el desarrollo de BIVORITA, el trabajo integrador final de la materia Programación 1, que consistió en implementar el clásico juego de la serpiente en lenguaje C. La consigna buscaba aplicar de forma integrada los temas vistos durante la cursada: listas enlazadas, matrices, manejo de memoria dinámica, paso de parámetros por puntero y persistencia de datos en archivos.

El trabajo se organizó en equipo de tres integrantes, dividiendo el desarrollo en módulos independientes (serpiente, tablero, visuales, teclado y jugadores) que después se integraron en un único programa a través de main.c. El objetivo de este documento es dejar por escrito el diseño, el funcionamiento y las decisiones tomadas durante el desarrollo, para que sirva como apoyo en la defensa oral del trabajo.

La idea principal era, además de cumplir la consigna, crear algo divertido desde el código hasta la ejecución y a su vez de fácil comprensión: con distintos niveles de dificultad, obstáculos, una interfaz visual clara por colores y un sistema de ranking persistente entre partidas. Esto implicó pensar no solo en que el código compilara y funcionara, sino también en la experiencia de quien juega y en que cada módulo fuera lo suficientemente independiente como para poder desarrollarse y probarse por separado antes de integrarlo con el resto.

2. Integrantes y participación en el trabajo

El trabajo se hizo de forma colaborativa entre los tres integrantes del grupo, dividiendo el proyecto en módulos para poder avanzar en paralelo. Usamos Git y GitHub como control de versiones, lo que nos permitió repartirnos las partes, subir los avances por separado y después ir integrando el código de a poco. A continuación, detallamos la participación de cada uno.

Julián Panini Suárez: Fue el responsable de la **lógica de la serpiente**, diseñando la lista enlazada dinámica y las funciones de movimiento y liberación de memoria (malloc y free). Durante el desarrollo, resolvió errores críticos de segmentación relacionados con el manejo de punteros. Asimismo, implementó el módulo de **control de teclado** (teclado.c), desarrollando la captura de teclas sin bloquear el juego mediante `_kbhit` y `_getch`, y la limpieza del búfer de entrada. También gestionó la configuración del entorno **MSYS2** para la compilación.

Jenaro Ceccacci: Se encargó del diseño lógico del tablero (matriz) y de la integración de todos los módulos en el main.c. Desarrolló la lógica de generación de manzanas y obstáculos, y la validación de límites del mapa. Un aporte fundamental fue la implementación del **sistema de persistencia y ranking** (jugadores.c), programando la lectura y escritura del archivo de texto donde se guardan los puntajes. También aplicó la **normalización de nombres usando aritmética ASCII**, evitando el uso de librerías externas para cumplir estrictamente con lo visto en clase.

Federico Camerini: Su aporte se centró en la **arquitectura y organización del proyecto**. Investigó y lideró la implementación de la **modularización** en archivos de cabecera (.h) y fuentes (.c), asegurando el uso correcto de **include guards** para evitar errores de redefinición. Además, realizó la investigación técnica sobre las librerías de sistema windows.h y conio.h, definiendo los parámetros para el manejo de colores, el ocultamiento del cursor y la optimización visual de la consola, lo que facilitó el trabajo de renderizado de sus compañeros.

3. Descripción general del funcionamiento del programa

BIVORITA es una versión del clásico juego de la serpiente, implementada para consola de Windows. El programa arranca mostrando un menú principal donde se puede elegir jugar o salir. Si se elige jugar, se pide seleccionar una dificultad (fácil, media o difícil). Una vez elegida la dificultad, se inicializa el tablero (una matriz de 20x40) y se crea la serpiente con un solo segmento. Se genera la posición inicial de la manzana y, si corresponde, los obstáculos. A partir de ahí, el programa funciona mediante un “game loop” central que se encarga de actualizar el estado del juego en cada fotograma, hasta que ocurre alguna colisión.

3.1 Flujo del programa

El ciclo de juego se puede resumir en los siguientes pasos, que se repiten en cada frame:

1. Se lee si el jugador apretó alguna tecla (W/A/S/D para moverse, X para salir), sin bloquear el juego mientras tanto.
2. Se calcula la próxima posición de la cabeza según la dirección actual.
3. Se revisa si esa posición coincide con la manzana, para saber si la serpiente va a crecer este turno.
4. Se verifica si la nueva posición choca contra un borde del tablero, contra el propio cuerpo o contra un obstáculo.
5. Si no hay colisión, la serpiente se mueve, se actualiza la matriz para reflejar el nuevo estado y, si comió, se genera una nueva manzana y aumenta el puntaje.
6. Se redibuja el tablero completo, incluso en el frame donde ocurre una colisión (para que se alcance a ver la marca del choque).
7. Se espera una pausa antes del siguiente frame; esa pausa es justamente lo que define la velocidad del juego.
8. Fin del juego:

Cuando ocurre una colisión, el juego termina: se marca la casilla del choque, se muestra el puntaje final y se libera toda la memoria de la serpiente. Recién ahí se pasa a la parte de ranking: se pide el nombre del jugador, se compara contra los puntajes guardados en un archivo de texto (creando el registro si es la primera vez que juega ese nombre, o actualizándolo si superó su récord anterior), se guarda el archivo actualizado y se muestra el ranking completo por pantalla.

3.2 Entorno de desarrollo y compilación

El programa se desarrolló íntegramente en Windows. Mientras que parte del equipo utilizó las herramientas estándar de VS Code, Julián configuró el entorno **MSYS2 (UCRT64)** para asegurar que el compilador gcc integrara correctamente las librerías nativas de Windows necesarias para el proyecto. El código se organizó de forma modular para facilitar el trabajo en paralelo y la detección de errores: `gcc -I include -o snake.exe main.c snake.c tablero.c visuales.c teclado.c jugadores.c`

4. Especificación y justificación de las estructuras de datos utilizadas

Para resolver el problema utilizamos distintas estructuras, investigando cuáles se aplicaban mejor a elementos dinámicos (Listas) o estáticos (Matrices) a lo largo de la partida.

4.1 Matriz de tablero

Representa el estado de cada casilla del mapa (vacío, cuerpo, manzana, obstáculo o colisión). Usamos una matriz porque el tablero tiene un tamaño fijo durante toda la partida (20 filas por 40 columnas) y necesitábamos poder consultar el estado de una casilla puntual en tiempo constante, por ejemplo, para saber si una posición está libre antes de generar la manzana, o si el próximo movimiento choca contra algo: `int tablero [FILAS][COLUMNAS]; // FILAS = 20, COLUMNAS = 40`

4.2 Serpiente (lista enlazada de Nodo)

Cada segmento del cuerpo es un nodo con su fila, su columna y un puntero al siguiente segmento. Elegimos una lista enlazada y no un arreglo porque la serpiente cambia de longitud constantemente: crece cada vez que come. Con un arreglo hubiéramos tenido que definir de entrada un tamaño máximo y, además, desplazar todos los elementos una posición cada vez que se mueve. Con la lista, mover la serpiente es simplemente agregar un nodo nuevo como cabeza y, si no comió, liberar el último nodo, sin tocar el resto de la memoria ni desperdiciar espacio reservado de más:

```
typedef struct Nodo {
    int fila;      int
columna;  struct Nodo
*siguiente;

} Nodo;

typedef struct {

    Nodo *cabeza;  int
longitud;

} Serpiente;
```

4.3 Manzana y Obstáculo

Son structs simples con fila y columna. La manzana es una sola, así que no hacía falta una lista; los obstáculos, en cambio, se guardan en un arreglo de tamaño fijo (MAX_OBSTACULOS), porque la cantidad se define al elegir la dificultad y no cambia durante la partida, a diferencia de la serpiente.

```
typedef struct {
    int fila;
    int columna;

} Manzana;

typedef struct {
    int fila;
    int columna;

} Obstaculo;
```

4.4 Jugador (lista enlazada, ranking)

Para el ranking usamos otra lista enlazada, esta vez de jugadores (nombre, puntaje máximo y puntero al siguiente). La elegimos porque no sabemos de antemano cuántos jugadores distintos se van a registrar con el tiempo; la lista permite ir agregando nodos sin un límite fijo, cargándolos y guardándolos línea por línea desde y hacia un archivo de texto.

```
typedef struct Jugador {
    char nombre[MAX_NOMBRE];
    int puntajeMaximo;
    struct Jugador *siguiente;
} Jugador;
```

4.5 Complejidad y eficiencia

La combinación de matriz y listas enlazadas nos permite tener lo mejor de las dos estructuras. Consultar si una casilla está ocupada por un obstáculo o una manzana en la matriz es una operación directa e inmediata, ya que se accede por índice sin tener que recorrer otros elementos. Por otro lado, el uso de la lista para la serpiente evita desperdiciar memoria reservando un tamaño máximo que tal vez nunca se use, y hace que mover el cuerpo sea eficiente: solo agregamos un nodo en la cabeza y, si no comió, liberamos el último de la cola

5. Detalle de las funciones principales implementadas

A continuación, se detallan las funciones más importantes de cada módulo, agrupadas según el archivo donde están implementadas.

5.1 Módulo *snake.c*

inicializarSerpiente(Serpiente *s, int filaInicial, int colInicial): recibe un puntero a la estructura Serpiente y las coordenadas donde va a nacer. Reserva memoria dinámica para el primer nodo con malloc, carga sus coordenadas y lo marca como cabeza única (siguiente = NULL). También inicializa longitud en 1. No devuelve nada: modifica directamente la estructura recibida por puntero.

moverSerpiente(Serpiente *s, int nuevaFila, int nuevaCol, int crecio): es el corazón del movimiento. Recibe la serpiente, la posición ya calculada para la nueva cabeza y un flag que indica si comió manzana ese turno. Reserva un nodo nuevo, lo carga con las coordenadas recibidas y lo engancha como nueva cabeza, apuntando su siguiente a la vieja cabeza. Si creció es 0, recorre la lista hasta encontrar el penúltimo nodo, libera el último y corta la lista ahí; si creció es 1, no borra nada y simplemente incrementa longitud.

colisionaConCuerpoLista(Serpiente *s, int fila, int col, int ignorarCola): función de consulta que devuelve 1 si la posición coincide con algún segmento del cuerpo, o 0 si no. El parámetro ignorarCola permite saltar la comparación contra el último nodo, que se usa cuando la serpiente no va a crecer ese turno: ese segmento se va a mover de todas formas, así que compararlo generaría un falso choque.

liberarSerpiente(Serpiente *s): recorre todos los nodos liberando la memoria uno por uno, guardando primero el puntero al siguiente para no perder la referencia después de hacer free, y deja la estructura en un estado consistente (cabeza en NULL, longitud en 0).

5.2 Módulo *tablero.c*

inicializarTablero(int tablero[FILAS][COLUMNAS]): recorre las FILAS por COLUMNAS casillas poniendo cada una en VACIO (0), ya que un arreglo no iniciado puede contener basura.

fueraDeLimites(int nuevaFila, int nuevaColumna): devuelve 1 si alguna coordenada cae fuera del rango válido de filas o columnas, 0 si es una posición válida. Se llama antes de tocar la matriz con esas coordenadas, para no acceder fuera de sus límites.

actualizarMatrizSerpiente(int tablero[FILAS][COLUMNAS], Serpiente *s): sincroniza la matriz con la lista enlazada después de cada movimiento. Primero borra todas las marcas de CUERPO viejas, sin tocar MANZANA ni OBSTACULO, y después recorre la lista completa marcando CUERPO en cada nodo vigente.

generarManzana / generarObstaculos: usan rand() dentro de un do-while para sortear coordenadas hasta encontrar una casilla libre, y las guardan tanto en la matriz (para que se dibujen) como, en el caso de los obstáculos, en el arreglo de structs Obstaculo, para poder consultarlos rápido sin recorrer la matriz completa.

colisionaConObstaculo(Obstaculo obstaculos[], int cantidad, int fila, int columna): recorre el arreglo de obstáculos, no la matriz, comparando contra la posición recibida, y devuelve 1 si hay coincidencia.

5.3 Módulo *visuales.c*

dibujoTablero(int tablero[FILAS][COLUMNAS], Serpiente *s, int puntaje): se llama una vez por frame. Limpia la pantalla, dibuja el borde superior, recorre fila por fila decidiendo qué carácter y color imprimir según el contenido de cada celda (colisión, cabeza, cuerpo, manzana, obstáculo o vacío) y finalmente dibuja el borde inferior y el puntaje.

setColor(int color): función auxiliar que llama a SetConsoleTextAttribute de la API de Windows para cambiar el color de texto y de fondo de todo lo que se imprima después.

ocultarCursor(): usa SetConsoleCursorInfo para esconder el cursor parpadeante, que sin esto quedaría tildando sobre el dibujo del tablero en cada frame.

5.4 Módulo *teclado.c*

leerTecla(Direccion *dirActual, int *gameOver): recibe dos punteros porque necesita modificar variables que viven en main(). Guarda la dirección con la que arrancó el frame antes de leer nada, para evitar que, si se procesan varias teclas en el mismo frame, se termine generando una reversa de 180 grados en dos pasos. Recorre con un while(_kbhit()) todas las teclas pendientes, y según la tecla (w, a, s, d o x) actualiza la dirección o marca gameOver en 1.

limpiarBufferTeclado(): vacía cualquier tecla pendiente en el búfer, por ejemplo, el salto de línea que deja un scanf, para que no se cuele en una lectura posterior.

5.5 Módulo *jugadores.c*

cargarRanking(void): abre el archivo de ranking en modo lectura; si no existe todavía, devuelve NULL sin error, ya que puede ser la primera vez que se juega. Si existe, lee línea por línea con `fscanf` y arma una lista enlazada con todos los jugadores guardados.

registrarPuntaje(Jugador *lista, char *nombre, int puntajeNuevo): busca si el nombre ya está en la lista; si está, actualiza el puntaje solo si el nuevo es mayor; si no está, crea un nodo nuevo y lo agrega al final. Devuelve la lista, que puede ser la misma o tener un nodo más.

guardarRanking / mostrarRanking / liberarJugadores: la primera sobrescribe el archivo completo escribiendo una línea por jugador; la segunda imprime por pantalla todos los jugadores en el orden en que están enlazados; la tercera libera toda la memoria de la lista, con el mismo patrón que `liberarSerpiente`.

normalizarNombre(char *nombre): convierte el nombre a mayúsculas modificando el mismo arreglo, usando aritmética simple sobre el código ASCII (restando 32 a cada letra minúscula), para que el ranking no distinga mayúsculas de minúsculas.

5.6 Módulo *main.c*

Es el módulo que conecta todo. Muestra el menú principal y el de dificultad, inicializa el tablero y la serpiente, y genera la manzana y los obstáculos iniciales. Después ejecuta el bucle de juego, que en los comentarios del código está dividido en pasos: (A) captura de teclas, (B) cálculo de la nueva posición de la cabeza, (C) verificación de colisiones y movimiento, (D) redibujado del tablero y (E) pausa según la velocidad. Al terminar el juego, se encarga de liberar la memoria de la serpiente y de gestionar todo el flujo del ranking: cargarlo, pedir el nombre, registrar el puntaje, guardarlo y mostrarlo por pantalla.

6. Dificultades encontradas durante el desarrollo y cómo fueron abordadas

Al ser un trabajo dividido en módulos y hecho por tres personas a la vez, tuvimos varias dificultades típicas de trabajar en simultáneo sobre el mismo código:

Gestión de versiones con Git: en más de una ocasión nos pisamos el código entre nosotros, sobre todo en el punto donde se conecta la lógica de la serpiente con el tablero y el main. Al principio subíamos cambios sin avisar bien qué archivo estaba manipulando cada uno, lo que generó conflictos de fusión (merge) complejos que debimos resolver manualmente.

Lógica de la serpiente: La lista enlazada fue el mayor desafío técnico. Julián detectó y resolvió varios errores de segmentación que ocurrían al intentar mover la cola; el problema era que intentábamos liberar el nodo antes de actualizar el puntero del nodo anterior. Además, Jenaro identificó un "caso límite" jugando: la serpiente chocaba contra su propia cola en el mismo instante en que esta se movía. Lo resolvimos agregando una condición para ignorar el último segmento en la verificación de colisiones cuando la serpiente no está creciendo.

Búfer de teclado: tuvimos un problema donde el juego “recordaba” teclas presionadas anteriormente, haciendo que la serpiente tenga movimientos involuntarios en ese momento. La solución fue implementar una limpieza explícita del búfer de teclado después de cada entrada por scanf, asegurándonos de que solo se procesara el input realmente actual.

Problemas de entorno: Tuvimos complicaciones con la configuración de conio.h en algunos equipos, lo que generaba errores de compilación extraños. Además, durante la integración final en el main.c, surgieron conflictos en Git debido a que Julián y Jenaro modificaron funciones similares para el chequeo de colisiones. Nos sirvió para aprender a usar ramas de desarrollo y a resolver conflictos de código de forma manual.

En general, la mayoría de las dificultades tuvieron que ver con la coordinación entre los tres al trabajar sobre el mismo proyecto al mismo tiempo, más que con el lenguaje C en sí mismo. Necesitamos revisar el código en conjunto antes de dar por cerrada cada parte.

7. Conclusión

El desarrollo de **Bivorita** nos permitió aplicar de manera concreta los conceptos de la materia: listas enlazadas, matrices, memoria dinámica, paso de parámetros por puntero y manejo de archivos, todo integrado en un mismo proyecto. Pudimos concretar un programa funcional respetando una lógica de juego sólida, donde cada uno de nosotros tuvo que tomar roles investigar y programar

Más allá de lo técnico, el mayor aprendizaje fue la organización grupal. Dividir el trabajo en módulos, usar Git para evitar la superposición de código, separar la investigación y darnos clases entre nosotros contándonos lo aprendido. En definitiva, este trabajo nos dejó una base sólida sobre cómo encarar un proyecto grupal, como modularizar correctamente un programa, la investigación necesaria, como separar responsabilidades, pensar la relación entre las estructuras antes de escribir el código y probar cada parte por separado antes de la integración final.

8. Anexo: controles y funcionamiento del ranking

A modo de referencia rápida para la defensa oral, se resumen acá los controles del juego y el criterio que usa el sistema de ranking:

Controles: W (arriba), A (izquierda), S (abajo), D (derecha) para mover la serpiente, y X para salir del juego en cualquier momento. No se permite invertir la dirección 180 grados en un solo paso (por ejemplo, ir a la derecha y pasar directamente a la izquierda), ya que eso equivaldría a que la cabeza chocara instantáneamente contra el segundo segmento del cuerpo.

Dificultades disponibles: fácil (sin obstáculos, velocidad más lenta), media (5 obstáculos) y difícil (15 obstáculos, velocidad más alta). A medida que la serpiente come, el juego se va acelerando levemente, hasta un piso mínimo de velocidad para que siga siendo jugable.

Ranking: al finalizar la partida se pide el nombre del jugador, que se normaliza a mayúsculas para evitar duplicados por mayúsculas o minúsculas. Si el nombre ya existía en el archivo de ranking, el puntaje se actualiza únicamente si el nuevo resultado superó al anterior; si es la primera vez que juega ese nombre, se agrega como un registro nuevo al final del archivo.